

PROMPT LIFECYCLE: VERSIONING · HOT-SWAP PER-PROMPT TRACE CORRELATION

Deep Research — All Options for AWS AgentCore Runtime
Phoenix / Arize AX · Langfuse · Braintrust · AppConfig · Config Bundles · Harness · CDK
Hotswap

GENAI Agent Foundry · GENAIAF-559 | June 2026

Epic	GENAIAF-559
Space	GENAI Agent Foundry · GENAIB-119
User Story	As an Observer, I want to iterate on prompts WITHOUT redeploying the agent
Scope	SDK-owned: versioning, trace correlation. TP dashboard/editing = Orange (out of scope)
New Context	AgentCore Config Bundles (May 2026) + Harness Versioning (GA June 18 2026) materially change the solution space
Date	June 2026

Executive Summary

GENAIAF-559 requests the minimum viable SDK capability for: (1) prompt versioning tied to agent config, (2) hot-swap — runtime prompt switching without agent redeploy, and (3) prompt version ID emitted in traces for Phoenix/AX correlation. This research documents every viable option available as of June 2026 and provides an implementation recommendation.

Critical new context since the ticket was written: AWS shipped two capabilities in April–June 2026 that directly address this epic without any external platform dependency:

- **AgentCore Configuration Bundles (Preview May 2026):** Immutable, versioned snapshots of model + system prompt + tool descriptions. Agent reads active config dynamically at runtime via AgentCore SDK — swapping a prompt is a config change, NOT a code change or redeploy.
- **AgentCore Harness (GA June 18 2026):** Managed agent loop where system prompt is a harness parameter. Prompt updates via UpdateHarness create a new immutable version. Named endpoints (PROD, STAGING) stay pinned until explicitly promoted. Per-invocation system prompt override available without touching harness defaults.
- **CDK Hotswap for AgentCore Runtime (GA January 2026):** cdk deploy --hotswap directly updates AgentCore Runtime container/S3 source without a full CloudFormation stack update — significantly faster than a standard redeploy.

This epic can be delivered with zero external PromptOps platform as an MVP. The question for the team is: does 'per-prompt trace correlation with Phoenix/AX' require external platform integration, or can AgentCore's native OTel + CloudWatch suffice?

Option Summary

Option A	RECOMMENDED MVP	AgentCore Config Bundles (Native) Zero external dependency. Hot-swap without redeploy. Version history. A/B via Gateway. Trace correlation via bundle ID in OTel spans. GA path imminent.
Option B	RECOMMENDED — Harness path	AgentCore Harness Versioning (Native) GA June 18 2026. UpdateHarness = new immutable version. Named endpoints = environment pinning. Per-invocation prompt override = instant hot-swap. No redeploy.
Option C	RECOMMENDED — Full lifecycle	Langfuse Prompt Registry + AppConfig SDK prompt fetch + version label (production). Trace linkage via Langfuse prompt object. AppConfig delivers hot payload. Best external platform for this use case.
Option D	OBSERVABILITY layer only	Arize AX / Phoenix + AppConfig Phoenix/AX does NOT provide hot prompt delivery. Use for trace correlation and online eval ONLY. Must pair with AppConfig or Langfuse for delivery.
Option E	MINIMUM VIABLE DELIVERY	AWS AppConfig alone (no registry) Fast, AWS-native, zero platform cost. No lineage or approval workflow. Suitable for teams not ready for external platform.
Option F	NOT TRUE HOT-SWAP	CDK Hotswap Deploy Reduces redeploy time from ~5min to ~30s. Does NOT eliminate deployment step. Valid for teams where AppConfig complexity is unwanted.
Option G	FUTURE STATE	Full external registry (Braintrust/MLflow) Best-in-class eval-gated CI/CD. Adds governance layer. More complex. Recommended for Phase 2 when lifecycle maturity is needed.

§1

The Three Requirements Decomposed

Breaking down the Jira epic into precise technical requirements before evaluating options.

Requirement 1: Prompt Versioning (git-based, tied to agent config)

The ticket specifies 'git-based' versioning. This means: prompts live in Git as the source of truth. The agent config (manifest, harness config, or appconfig profile) references a specific prompt version, not an inline string. This gives you diff history, blame, PR review, and rollback via git.

What it does NOT mean: Git alone is the runtime delivery mechanism. Git is the authoring source; something else (AppConfig, Langfuse SDK, AgentCore Config Bundle) serves the prompt at runtime.

Requirement 2: Hot-Swap — Runtime Prompt Switching Without Redeploy

This is the technically hardest requirement. 'Without redeploy' means: a running AgentCore Runtime container should pick up a new prompt version **without the container being rebuilt, re-pushed to ECR, or re-deployed via CDK/CloudFormation**. The agent process must continue serving requests throughout the update.

What counts as hot-swap?	Verdict
AppConfig sidecar poll (45s interval) — new config served to same container	■ TRUE hot-swap
AgentCore Config Bundle active bundle switch via SDK at invocation time	■ TRUE hot-swap (per-invocation)
AgentCore Harness UpdateHarness + named endpoint update	■ TRUE hot-swap for Harness path
Langfuse SDK fetch with label=production updated in UI	■ TRUE hot-swap (next invocation)
CDK deploy --hotswap (skips CloudFormation, directly updates Runtime)	■■ FASTER deploy, NOT true hot-swap
Rebuilding container + cdk deploy (standard)	■ NOT hot-swap — full redeploy
Hardcoded prompt in source code	■ Requires redeploy for any change

Requirement 3: Prompt Version ID Emitted in Traces

Phoenix and Arize AX display traces as spans. To correlate quality regression to a specific prompt version, every LLM invocation span must carry the prompt version as a span attribute. The target attributes are:

```
# OpenInference / GenAI OTel semantic conventions
gen_ai.prompt.id      = 'customer-support-system' # prompt name/slug
gen_ai.prompt.version = '2.3.1'                  # semver or bundle ID
gen_ai.prompt.hash    = 'sha256:abc123...'       # tamper detection
gen_ai.prompt.label   = 'production'             # Langfuse label or bundle tag

# These appear as filterable dimensions in Phoenix + Arize AX trace UI
# Arize AX: filter by prompt.version to compare quality across versions
# Phoenix: group traces by prompt.version for offline eval analysis
```

AgentCore's built-in OTel (ADOT) does NOT automatically emit prompt version attributes. The agent code or SDK wrapper must set these attributes explicitly on the generation span. This is a required SDK deliverable for the epic regardless of which hot-swap option is chosen.

§2

Option A: AgentCore Configuration Bundles

Native AWS hot-swap for prompts — Preview May 2026, tracking GA. The most important new capability for this epic.

What Configuration Bundles Are

Configuration Bundles are immutable, versioned snapshots of an agent's configuration keyed by AgentCore Runtime ARN. Each bundle captures: model ID, system prompt, tool descriptions. The agent reads its active bundle dynamically at runtime through the AgentCore SDK — swapping a prompt or model is a configuration change, NOT a code change.

This is the native AWS answer to the hot-swap requirement. It decouples configuration from code, enabling prompt changes without redeploying application code — provided the runtime is designed to read configuration dynamically via the AgentCore SDK.

How Configuration Bundles Work

Step 1: Design prompt in Git	Author writes/updates system prompt in Git repo. PR review, peer approval, CI linting all run against the Git version.	▼
Step 2: Create bundle	agentcore bundle create --name 'v2-improved-prompt' --system-prompt 'prompts/cx-v2.txt' --model-id claude-sonnet-4-6. Bundle is immutable and versioned after creation.	▼
Step 3: Agent reads bundle at invocation	AgentCore SDK reads active bundle configuration at invocation time. Agent calls get_active_config() — returns current model, system prompt, tool descriptions.	▼
Step 4: Swap active bundle	Update the active bundle reference via AgentCore control API or CLI — no container rebuild, no redeploy. In-flight sessions continue with previous bundle if session-pinned.	▼
Step 5: A/B via Gateway	Create two bundles (current + candidate). Configure AgentCore Gateway to split traffic. Each session gets bundle A or B. Statistical significance tracked.	▼
Step 6: Promote winner or rollback	If bundle B wins A/B test → update active bundle to B. If B regresses → revert active bundle to A. Both are zero-redeploy config changes.	

SDK Integration Pattern

```
# agent/main.py – reads active config bundle at invocation time
from bedrock_agentcore import get_active_config
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

def invoke(session_id: str, user_input: str) -> str:
    # Get active config bundle (cached, low latency)
    config = get_active_config() # Returns model, system_prompt, tool_descriptions
    bundle_id = config.bundle_id # e.g. 'bundle-cx-v2-abc123'
    bundle_ver = config.version # e.g. '2'
    system_prompt = config.system_prompt

    # Emit bundle metadata as OTel span attributes (CRITICAL for Phoenix/AX)
    with tracer.start_as_current_span('agent.invoke') as span:
        span.set_attribute('gen_ai.prompt.id', 'customer-support-system')
        span.set_attribute('gen_ai.prompt.version', bundle_ver)
        span.set_attribute('gen_ai.bundle.id', bundle_id)
        span.set_attribute('session.id', session_id)

    response = bedrock_client.converse(
        modelId=config.model_id,
        system=[{'text': system_prompt}],
        messages=[{'role': 'user', 'content': [{'text': user_input}]}]
    )
    return response['output']['message']['content'][0]['text']
```

Bundle CLI Commands

```
# Create a bundle from prompt file in Git
agentcore bundle create \
  --name 'cx-system-v2' \
  --system-prompt 'prompts/cx-v2.txt' \
  --model-id 'us.anthropic.claude-sonnet-4-6' \
  --runtime-id my-runtime-abc123

# List all bundles for a runtime
agentcore bundle list --runtime-id my-runtime-abc123

# Switch active bundle (hot-swap – no redeploy!)
agentcore bundle activate \
  --bundle-id bundle-cx-v2-abc123 \
  --runtime-id my-runtime-abc123

# Rollback to previous bundle
agentcore bundle activate \
  --bundle-id bundle-cx-v1-xyz789 \
  --runtime-id my-runtime-abc123

# A/B test via Gateway: split traffic 90/10 between bundles
agentcore gateway ab-test create \
  --control-bundle bundle-cx-v1-xyz789 \
  --candidate-bundle bundle-cx-v2-abc123 \
  --split 90/10 --runtime-id my-runtime-abc123
```

What Config Bundles Do and Don't Provide

Capability	Config Bundles Status	Notes
Hot-swap without redeploy	■ YES — core purpose	Active bundle switch takes effect on next invocation
Versioning	■ YES — immutable versions	Each bundle is immutable after creation
Model + prompt + tool desc	■ YES	All three in single bundle snapshot
A/B testing	■ YES — via AgentCore Gateway	Traffic split with per-session consistency
Trace correlation	■■ Manual — bundle ID in span	Must be added by SDK code (shown above)
Semantic versioning (semver)	■ NO	Bundle IDs, not semver. Layer semver on top externally.

Approval workflow	■ NO	No built-in human approval gate
Lineage / dependency graph	■ NO	No cross-bundle dependency tracking
Phoenix/AX integration	■■ Via OTEL span attributes	Bundle ID visible in Phoenix/AX if emitted as span attr
GA status	Preview May 2026	GA timeline: watch AWS announcements. API subject to change.

§3

Option B: AgentCore Harness Versioning

GA June 18 2026 — managed agent loop with immutable versioning, named endpoints, and per-invocation overrides.

What the AgentCore Harness Provides

The Harness (GA June 18, 2026) is a managed agent loop. You declare: model, system prompt, tools, memory, limits. AgentCore runs the orchestration loop (powered by Strands Agents under the hood). Every `UpdateHarness` creates a new immutable version. Named endpoints (PROD, STAGING) stay pinned until explicitly promoted.

For the hot-swap use case, the Harness offers **three distinct mechanisms**, each with different tradeoffs:

Mechanism 1: UpdateHarness + Endpoint Promotion

Change the system prompt → new harness version created. Promote named PROD endpoint to new version = instant switch for all new sessions. Previous version stays available for rollback.

```
# Step 1: Update harness with new system prompt (creates V3)
aws bedrock-agentcore-control update-harness \
  --harness-id MyHarness-abc123 \
  --system-prompt '[{"text": "You are an improved customer support agent v3..."}]'
# This creates an immutable V3 snapshot. PROD endpoint still points to V2.

# Step 2: Promote PROD endpoint to V3 (hot-swap – no container rebuild)
aws bedrock-agentcore-control update-harness-endpoint \
  --harness-id MyHarness-abc123 \
  --endpoint-name production-endpoint \
  --target-version 3 \
  --description 'Promoting improved CX prompt to production'

# Rollback to V2: one command
aws bedrock-agentcore-control update-harness-endpoint \
  --harness-id MyHarness-abc123 \
  --endpoint-name production-endpoint \
  --target-version 2 --description 'Rollback to V2'
```

Mechanism 2: Per-Invocation System Prompt Override

The Harness config-based model allows overriding the system prompt at invoke time **without changing the harness defaults**. The harness resource stays unchanged; only that call uses the overrides. This is instant hot-swap per invocation — no version bump needed.


```
# Python SDK: per-invocation system prompt override
import boto3
from opentelemetry import trace

client = boto3.client('bedrock-agentcore', region_name='us-east-1')
tracer = trace.get_tracer(__name__)

def invoke_with_prompt(harness_arn: str, session_id: str,
                      prompt_text: str, prompt_version: str,
                      user_input: str) -> str:
    with tracer.start_as_current_span('harness.invoke') as span:
        # Emit prompt version for Phoenix/AX trace correlation
        span.set_attribute('gen_ai.prompt.id', 'cx-system')
        span.set_attribute('gen_ai.prompt.version', prompt_version)
        span.set_attribute('session.id', session_id)

        response = client.invoke_harness(
            harnessArn=harness_arn,
            runtimeSessionId=session_id,
            # Override system prompt for this call only – harness defaults intact
            systemPrompt=[{'text': prompt_text}],
            payload={'prompt': user_input}
        )
    return response['output']
```

Mechanism 3: Variants (A/B Testing Endpoints)

Route traffic between two harness versions via different endpoints. AgentCore Gateway handles traffic splitting with per-session consistency and statistical significance reporting.

```
# Create STAGING endpoint pinned to V3 (candidate)
aws bedrock-agentcore-control create-harness-endpoint \
  --harness-id MyHarness-abc123 \
  --endpoint-name staging-v3 --target-version 3

# A/B: 90% to production (V2), 10% to staging-v3
# Configure via AgentCore Gateway A/B routing
# When V3 wins statistical significance -> promote:
aws bedrock-agentcore-control update-harness-endpoint \
  --harness-id MyHarness-abc123 \
  --endpoint-name production-endpoint --target-version 3
```

Harness vs Runtime — Which Path to Take?

Factor	Use Harness	Use Runtime (code-based)
Agent architecture	Simple single-agent loop (model → tools → response)	Complex LangGraph DAG, multi-agent, custom orchestration
Prompt update mechanism	UpdateHarness (no code change, no container rebuild)	Config Bundles or AppConfig (config change, no rebuild)
Hot-swap speed	Endpoint switch — seconds	Bundle activate or AppConfig — seconds to next poll
Per-invocation override	■ Native (systemPrompt on InvokeHarness)	■ Via Config Bundle per-invocation override
Version history	■ Immutable harness versions V1, V2, V3...	■ Immutable bundles; not sequential version numbers
A/B testing	■ Named endpoints + Gateway split	■ Gateway A/B with different bundle IDs
Framework flexibility	Fixed — Strands Agents under the hood	Any framework: LangGraph, CrewAI, Strands, custom
Operational complexity	Lower — AWS manages loop	Higher — you manage loop + SDK integration
Export to code	■ agentcore export harness → Strands code	Already code

For GENAIAF-559: If the team's agents are not complex LangGraph DAGs (i.e., standard model + tools), the Harness path is the simplest implementation of hot-swap versioning. UpdateHarness + endpoint promotion satisfies all three requirements with zero external dependencies.

§4

Option C: Langfuse Prompt Registry + AppConfig Delivery

Best external platform option — proven AgentCore integration, MIT license, full lifecycle management.

Why Langfuse for This Epic

Langfuse has a documented, official integration with Amazon Bedrock AgentCore (published November 2025, AWS blog December 2025). The integration is OTel-native and handles the ADOT conflict that affects other platforms. Langfuse's prompt management is purpose-built for the 'iterate without redeploy' use case — promote a new version to the 'production' label in the UI, and the running agent picks it up on the next SDK fetch.

Architecture: Two Layers

Layer	Tool	Responsibility
Governance & Registry	Langfuse	Versioning, labels (production/staging), Jinja2 template compilation, trace linkage, eval history
Hot Delivery	AWS AppConfig	Sub-5ms cached delivery to agents, canary deployment strategies, automatic rollback, local sidecar
Cache & Invalidation	ElastiCache Redis	Sub-ms cache hit, pub/sub invalidation when AppConfig updates
Observability	Phoenix (dev) / Arize AX (prod)	OTel trace ingestion, prompt version filtering, online eval

Full Implementation — Langfuse + AppConfig + AgentCore

```
# 1. Create and publish prompt in Langfuse
from langfuse import Langfuse
langfuse = Langfuse() # uses LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY env vars

# Author publishes new version (can be done from Langfuse UI too)
langfuse.create_prompt(
    name='customer-support-system',
    prompt='You are {{persona}} for {{company}}. {{safety_clause}}',
    config={'model': 'claude-sonnet-4-6', 'temperature': 0.3},
    labels=['production'], # promotes immediately to production label
    version=3,             # semver or auto-increment
)

# 2. CI/CD: sync production prompt to AppConfig for hot delivery
# (Lambda triggered by Langfuse webhook on label change)
def sync_to_appconfig(prompt_name: str, label: str = 'production'):
    prompt_obj = langfuse.get_prompt(prompt_name, label=label)
    new_config = {
        'template': prompt_obj.prompt,
        'version': str(prompt_obj.version),
        'config': prompt_obj.config,
        'langfuse_id': prompt_obj.id,
    }
    appconfig_client.create_hosted_configuration_version(
        ApplicationId=APPCONFIG_APP_ID,
        ConfigurationProfileId=PROFILE_ID,
        Content=json.dumps(new_config).encode(),
        ContentType='application/json',
    )
    appconfig_client.start_deployment(
        ApplicationId=APPCONFIG_APP_ID,
        EnvironmentId=PROD_ENV_ID,
        DeploymentStrategyId=CANARY_STRATEGY_ID, # e.g. 5% canary
        ConfigurationProfileId=PROFILE_ID,
        ConfigurationVersion='latest',
    )
```

```
# 3. AgentCore agent: fetch from AppConfig sidecar + emit trace attrs
import requests, json, time
from opentelemetry import trace

APPCONFIG_URL = 'http://localhost:2772/applications/prompts/environments/prod/configurations'
OTEL_EXPORTER_ENDPOINT = 'https://cloud.langfuse.com/api/public/otel'

tracer = trace.get_tracer(__name__)
_cache = {} # prompt_name -> {content, ts}

def get_prompt(name: str, ttl: int = 45) -> dict:
    cached = _cache.get(name)
    if cached and (time.time() - cached['ts']) < ttl:
        return cached['data']
    resp = requests.get(f'{APPCONFIG_URL}/{name}', timeout=1.0)
    data = resp.json()
    _cache[name] = {'data': data, 'ts': time.time()}
    return data

def render_prompt(template: str, variables: dict) -> str:
    import jinja2
    return jinja2.Template(template).render(**variables)

def invoke(session_id: str, user_input: str, context: dict) -> str:
    prompt_cfg = get_prompt('customer-support-system')
    prompt_ver = prompt_cfg['version']
    langfuse_id = prompt_cfg['langfuse_id']
    system_prompt = render_prompt(prompt_cfg['template'], context)

    with tracer.start_as_current_span('llm.invoke') as span:
        # CRITICAL: emit prompt version for Phoenix/AX correlation
        span.set_attribute('gen_ai.prompt.id', 'customer-support-system')
        span.set_attribute('gen_ai.prompt.version', prompt_ver)
        span.set_attribute('langfuse.prompt.id', langfuse_id)
        span.set_attribute('session.id', session_id)

    response = bedrock_client.converse(
        modelId=prompt_cfg['config']['model'],
        system=[{'text': system_prompt}],
        messages=[{'role': 'user', 'content': [{'text': user_input}]}]
    )
    return response['output']['message']['content'][0]['text']
```

Langfuse AgentCore OTel Integration (Critical Detail)

AgentCore's built-in ADOT (AWS Distro for OpenTelemetry) conflicts with Langfuse SDK callbacks. You must disable ADOT and configure your own OTel exporter. This is documented by both Langfuse and AWS:

```
# Environment variables for AgentCore agent deployment
# Set these in agentcore.json or CDK stack environment

DISABLE_ADOT_OBSERVABILITY=true # Disable AWS built-in tracing

# Configure OTel to export to Langfuse
OTEL_EXPORTER_OTLP_ENDPOINT=https://cloud.langfuse.com/api/public/otel
OTEL_EXPORTER_OTLP_HEADERS=Authorization=Basic $(echo -n 'pk-lf-xxx:sk-lf-xxx' | base64)

# For HIPAA/self-hosted Langfuse:
OTEL_EXPORTER_OTLP_ENDPOINT=https://langfuse.internal.company.com/api/public/otel

# For Arize Phoenix (dev/local):
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:6006/v1/traces

# For Arize AX (production):
OTEL_EXPORTER_OTLP_ENDPOINT=https://otlp.arize.com/v1
OTEL_EXPORTER_OTLP_HEADERS=api_key=, space_key=
```

Langfuse Prompt → Trace Linkage

Langfuse natively supports linking traces to prompt versions. When the Langfuse SDK's `get_prompt()` is used (instead of AppConfig), the prompt object includes a `link()` method that automatically stamps the trace with the

prompt version ID:

```
# Alternative: direct Langfuse SDK fetch (no AppConfig)
# Best for dev/test; AppConfig preferred for production performance
from langfuse import Langfuse
from langfuse.decorators import observe, langfuse_context

langfuse = Langfuse()

@observe()
def invoke(session_id: str, user_input: str, context: dict) -> str:
    # Fetch prompt + auto-link to trace
    prompt_obj = langfuse.get_prompt('customer-support-system', label='production')
    compiled = prompt_obj.compile(**context) # Jinja2 variable injection

    # Emit version as OTel span attribute for Phoenix/AX
    langfuse_context.update_current_observation(
        metadata={'prompt_version': str(prompt_obj.version),
                  'prompt_id': prompt_obj.id}
    )

    response = bedrock_client.converse(
        modelId='us.anthropic.claude-sonnet-4-6',
        system=[{'text': compiled}],
        messages=[{'role': 'user', 'content': [{'text': user_input}]}]
    )
    # Langfuse auto-links this trace to prompt_obj.version in the UI
    return response['output'][0]['message']['content'][0]['text']
```

S5

Option D: Phoenix / Arize AX Role

What Phoenix and AX actually provide for this epic — and what they don't.

The Core Clarification

Phoenix and Arize AX are **observability and evaluation platforms**. They do NOT provide hot prompt delivery to running agents. The Jira ticket scopes the TP dashboard/editing as Orange (out of scope). That means Phoenix/AX contribute to this epic only at the **trace correlation** layer — not the versioning or delivery layer.

What Phoenix/AX Provides for GENAIAF-559	What Phoenix/AX Does NOT Provide
Ingests OTel traces with prompt version span attributes	Hot-swap prompt delivery to agents
Filters traces by prompt.version — compare quality across versions	Prompt registry with semver or approval workflow
LLM-as-judge evaluations on production traffic (AX online eval)	Session version pinning logic
Prompt IDE for iterating on prompt content (AX only)	AppConfig-style canary deployment
Evaluator Hub with commit-level version control (AX Jan 2026)	Per-session rollback capability
Alert when quality degrades after a new prompt version	Git-tied versioning

How to Make Phoenix/AX Work with AgentCore

The ADOT conflict must be resolved. AgentCore injects its own OTel provider at runtime, which can intercept or bypass SDK HTTP calls. The fix:

```
# For Phoenix (development / self-hosted):
DISABLE_ADOT_OBSERVABILITY=true
OTEL_EXPORTER_OTLP_ENDPOINT=http://your-phoenix-host:6006/v1/traces

# Initialize Phoenix OTel in agent startup:
import phoenix as px
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter

exporter = OTLPSpanExporter(endpoint='http://phoenix-host:6006/v1/traces')
provider = TracerProvider()
provider.add_span_processor(BatchSpanProcessor(exporter))
trace.set_tracer_provider(provider)

# LiteLLM integration: use Arize Phoenix prompts via litellm
import litellm
response = litellm.completion(
    model='bedrock/us.anthropic.claude-sonnet-4-6',
    prompt_id='UHJvbXB0VmVyc2lvbjox',      # Phoenix prompt ID
    prompt_integration='arize_phoenix',
    api_key=os.environ['PHOENIX_API_KEY'],
    api_base='https://app.phoenix.arize.com/s/your-workspace',
    prompt_variables={'company': 'Acme Corp', 'product': 'Widget Pro'},
)
# Phoenix automatically links trace to the prompt version used
```

Phoenix Prompt Versioning Capabilities (Honest Assessment)

- Phoenix has basic prompt versioning in its UI — create versions, compare diffs, playground testing.
- Phoenix does NOT have semver, approval workflows, lineage graphs, or dependency tracking.
- The litellm integration above allows Phoenix prompts to be fetched at runtime — but NOT via AppConfig caching. Latency is 50–200ms per fetch without caching. Cache not built in.
- For production hot-swap, Phoenix prompts should be synced to AppConfig as an intermediary, not fetched directly at each invocation.
- Arize AX's Prompt IDE is for experimentation, not production delivery. Same constraint applies.

Recommended Role for Phoenix/AX in GENAIAF-559: Phoenix = development-time tracing and offline eval. Arize AX = production online evaluation and quality monitoring. Neither should own the hot-swap delivery mechanism. AppConfig or Config Bundles deliver prompts; Phoenix/AX receive traces showing which version ran.

§6

Option E: AppConfig Alone |
Option F: CDK Hotswap

AWS-native options with no external registry dependency.

Option E: AWS AppConfig as Sole Prompt Store

AppConfig is the simplest AWS-native path to hot-swap. No external platform needed. Prompt content stored in AppConfig as JSON. Agent fetches via localhost sidecar (< 5ms). AppConfig handles deployment strategies, canary rollout, and automatic rollback.

What you give up: No semantic versioning, no lineage, no approval workflow, no semantic search, no evaluation history. This is appropriate for teams at PromptOps Maturity Level 1–2.

```
# AppConfig prompt schema
{
  'prompt_id':      'customer-support-system',
  'version':        '2.3.1',                # Manual semver – no enforcement
  'template':       'You are {{persona}} for {{company}}...',
  'model_id':       'us.anthropic.claude-sonnet-4-6',
  'updated_at':     '2026-06-20T09:00:00Z',
  'updated_by':     'engineer@company.com'
}

# Agent fetches via AppConfig Lambda extension sidecar
def get_prompt(name: str) -> dict:
    resp = requests.get(
        f'http://localhost:2772/applications/agent-prompts'
        f'/environments/production/configurations/{name}',
        timeout=0.5 # < 5ms from cache
    )
    return resp.json()

# Emit version for Phoenix/AX
cfg = get_prompt('customer-support-system')
span.set_attribute('gen_ai.prompt.version', cfg['version'])

# Update prompt: no agent restart
# 1. Edit JSON in AppConfig console or via CLI
# 2. Start deployment with canary strategy
# 3. Agent picks up new version within next poll cycle (default 45s)
```

AppConfig Deployment Strategies

Strategy	Roll-out Speed	Risk	Use Case for Prompts
Instant (AllAtOnce)	Immediate	High	Typo fixes, metadata-only changes, safety patches
Linear 20%/2min	~10 min total	Medium	Standard MINOR version changes
Canary 5% for 15min	~15 min	Low	Feature additions, new capabilities
Exponential ramp	30–60 min	Very Low	MAJOR behavioral changes
Bake time: 30 min	Manual gate	Lowest	High-risk regulated context changes

Option F: CDK Hotswap Deployments (GA January 2026)

AWS CDK v2.1102.0+ (January 2026) supports --hotswap flag for AgentCore Runtime. Instead of updating the CloudFormation stack, CDK directly updates the AgentCore Runtime resource using the AWS SDK. This reduces deployment time from ~5 minutes to ~30 seconds.

CDK hotswap is NOT true hot-swap in the sense of the Jira epic. It still requires rebuilding and redeploying the agent. The difference is speed (30s vs 5min), not elimination of the redeploy step. An agent receiving a request during the 30-second window will experience a brief interruption.

```
# CDK hotswap – faster deploy, but still a deploy
cdk deploy --hotswap # Skips CloudFormation, directly updates AgentCore Runtime

# Supported in CDK v2.1102.0+
# Supports both ECR container image and S3 code (Python) updates

# Use case: when AppConfig/Config Bundles complexity is unwanted
# and 30s deploy windows are acceptable for the team's SLA

# NOT suitable if:
# - Agents run for > 30 seconds (ongoing tasks will be interrupted)
# - You need zero-downtime semantics
# - You need session version pinning for active conversations
```

Factor	AppConfig (Option E)	CDK Hotswap (Option F)
Zero-downtime?	■ True zero-downtime	■ 30s window, not zero-downtime
Session pinning?	■ With custom implementation	■ No — new container
Rollback time?	< 5s (AppConfig switch)	~30s (another hotswap deploy)
External tooling?	AppConfig (AWS-native)	CDK (already in use)
Prompt history?	AppConfig version history	Git commit history
A/B testing?	■ AppConfig + canary strat.	■ Requires dual deployments
Best for?	Production, regulated, long-running agents	Dev/test speed, simple pipelines

s7

Per-Prompt Trace Correlation —
Complete Implementation

How to emit prompt version ID in every span for Phoenix and Arize AX — regardless of delivery option.

The OTEL Span Attribute Standard

OpenInference (the OTEL semantic conventions standard for LLM spans, maintained by Arize AI) defines the span attributes for prompt correlation. These are the attributes Phoenix and Arize AX understand natively and use for filtering, grouping, and quality comparison.

```
# Target OTEL span attributes for prompt version correlation
# Based on OpenInference conventions (openinference.semconv)

# Core prompt identity
gen_ai.prompt.id      = 'customer-support-system'    # prompt slug/name
gen_ai.prompt.version = '2.3.1'                     # semver / bundle ID

# Source-specific attributes (add whichever applies)
langfuse.prompt.id    = 'cGrFYz...'                 # Langfuse prompt UUID
agentcore.bundle.id   = 'bundle-cx-v2-abc123'        # AgentCore bundle ID

# Session context for multi-turn correlation
session.id            = 'sess-uuid-v4'               # AgentCore session ID
user.id               = 'user-abc'                   # (if applicable)

# Additional quality signal context
deployment.label      = 'production'                  # or 'canary', 'staging'
ab_test.variant       = 'A'                           # or 'B', for A/B splits
```

Universal PromptClient SDK Class

Regardless of which delivery option the team chooses (Config Bundles, AppConfig, or Langfuse), a single PromptClient wrapper abstracts the delivery mechanism and handles trace emission:

```
# sdk/prompt_client.py (part 1)
import json, time, hashlib, requests
from opentelemetry import trace
from dataclasses import dataclass
from typing import Optional
@dataclass
class PromptResult:
    content: str; prompt_id: str; version: str
    source_id: Optional[str]=None; sha256: Optional[str]=None
class PromptClient:
    def __init__(self, source='appconfig', ttl=45):
        self.source=source; self.ttl=ttl; self._cache={}
        self.tracer=trace.get_tracer(__name__)
    def get(self, prompt_id, variables=None):
        raw=self._fetch(prompt_id)
        content=self._render(raw['template'],variables or {})
        sha=hashlib.sha256(content.encode()).hexdigest()[:16]
        return PromptResult(content=content,prompt_id=prompt_id,
            version=raw.get('version','unknown'),
            source_id=raw.get('source_id'),sha256=sha)
    def instrument_span(self, span, r):
        span.set_attribute('gen_ai.prompt.id',r.prompt_id)
        span.set_attribute('gen_ai.prompt.version',r.version)
        span.set_attribute('gen_ai.prompt.sha256',r.sha256)
        if r.source_id:
            span.set_attribute('gen_ai.prompt.source_id',r.source_id)
```

```
# sdk/prompt_client.py (part 2)
    def _fetch(self, prompt_id):
        c=self._cache.get(prompt_id)
        if c and (time.time()-c['ts']) < self.ttl:
            if self.source=='appconfig': data=self._fetch_ac(prompt_id)
            elif self.source=='langfuse': data=self._fetch_lf(prompt_id)
            elif self.source=='bundle': data=self._fetch_bnd(prompt_id)
            self._cache[prompt_id]={ 'data':data, 'ts':time.time()}
            return data
    def _fetch_ac(self,name):
        r=requests.get(f'http://localhost:2772/applications/agent-prompts'
            f'/environments/production/configurations/{name}',timeout=1.0)
        return r.json()
    def _fetch_lf(self,name):
        from langfuse import Langfuse
        o=Langfuse().get_prompt(name,label='production')
        return {'template':o.prompt,'version':str(o.version),'source_id':o.id}
    def _fetch_bnd(self,name):
        from bedrock_agentcore import get_active_config
        c=get_active_config()
        return {'template':c.system_prompt,'version':c.version,'source_id':c.bundle_id}
    def _render(self,template,variables):
        import jinja2; return jinja2.Template(template).render(**variables)
# Usage:
client=PromptClient(source='appconfig',ttl=45)
def invoke(session_id,user_input,ctx):
    prompt=client.get('customer-support-system',ctx)
    with client.tracer.start_as_current_span('llm.invoke') as span:
        client.instrument_span(span,prompt)
        span.set_attribute('session.id',session_id)
```

What This Looks Like in Phoenix and Arize AX

Platform	Where Prompt Version Appears	What You Can Do With It
Arize Phoenix	Span attribute pane on each trace span. Visible as 'gen_ai.prompt.version'	Filter traces by version, group spans, compare offline eval datasets by version
Arize AX	Span attributes + dimension in online eval dashboard. Prompt version filterable column	Alert when quality drops below threshold for a specific version. Compare V2 vs V3 quality online.
Langfuse	Prompt version linked in trace UI (when using Langfuse SDK fetch). 'Prompt' tab on generation span	Version diff view, compare quality metrics per version, click from trace to prompt definition
CloudWatch	Custom EMF metric or log attribute. Query via Insights / Athena	Cost and token usage per prompt version. Latency comparison. Alert on version-specific regression.

S8

Session Version Pinning

Ensuring active sessions don't experience behavioral changes mid-conversation when a new prompt is deployed.

Hot-swap raises a consistency question: if a conversation is on message turn 4 when prompt V16 deploys, should messages 5+ use V15 or V16? For most regulated enterprise contexts, the answer is V15 — session consistency is non-negotiable. This section shows how to implement it for each option.

Pattern: Store Prompt Version in AgentCore Memory at Session Creation

```
# On session creation – pin prompt version in AgentCore Memory
from bedrock_agentcore.memory import MemoryClient

memory = MemoryClient(memory_id=os.environ['AGENTCORE_MEMORY_ID'])

def create_session(user_id: str) -> dict:
    current_prompt = client.get('customer-support-system') # PromptClient
    session_id     = str(uuid.uuid4())

    # Write pinned version to AgentCore Memory (survives microVM restart)
    memory.put(
        namespace=f'session:{session_id}',
        key='pinned_prompts',
        value=json.dumps({
            'customer-support-system': current_prompt.version,
            'safety-guardrails':      safety_prompt.version,
            'pinned_at':              datetime.utcnow().isoformat(),
        }),
        ttl_seconds=86400 # 24h TTL for session
    )
    return {'session_id': session_id, 'prompt_version': current_prompt.version}

def get_prompt_for_session(session_id: str, prompt_id: str) -> PromptResult:
    record = memory.get(namespace=f'session:{session_id}', key='pinned_prompts')
    if record:
        pinned = json.loads(record)
        version = pinned.get(prompt_id)
        if version:
            # Fetch the specific pinned version (not 'production' label)
            return client.get_version(prompt_id, version=version)
    return client.get(prompt_id) # Fallback to current production
```

Session Pinning Strategies

Strategy	Behavior	Use Case
Pin at session creation (RECOMMENDED)	Version written to AgentCore Memory at first turn. All subsequent turns use pinned version.	Regulated context, long-running tasks, financial/healthcare
No pinning (rolling)	Every turn fetches current production prompt. Behavioral change can occur mid-conversation.	Simple FAQ bots, single-turn agents, non-regulated
Checkpoint migration	At natural pause points (e.g. topic change), session migrates to new version.	Conversational agents with clear topic boundaries
Safety override flag	Safety patches bypass session pinning — safety_patch=true in AppConfig deploys to ALL sessions immediately.	Critical safety updates only
Tenant-controlled	Enterprise tenant chooses migration window. Prompt Gateway respects tenant's active version policy.	Multi-tenant SaaS, enterprise customers with compliance needs

§9

Decision Matrix — All Options Compared

Scored against the three Jira requirements plus enterprise readiness dimensions.

Scoring Criteria for GENAIAF-559

Each option scored 1–5 on the six most relevant dimensions for this specific epic. Weight reflects Jira minimal delivery requirements.

Dimension	Weight	Why This Matters for GENAIAF-559
Git-based versioning (R1)	20%	Prompts must be source-controlled in Git. Version history, diff, PR review.
True hot-swap without redeploy (R2)	25%	Core requirement. Running agent must pick up new prompt without restart.
Prompt version in traces (R3)	20%	Phoenix/AX must be able to filter traces by prompt version.
Session version pinning	15%	Active sessions must not change behavior mid-conversation.
Implementation effort (inverse)	12%	Simpler = better for MVP scope. SDK primitives only.
Operational complexity	8%	On-call burden, failure modes, monitoring needs.

Decision Matrix

Option	R1 Git Version	R2 Hot-S wap	R3 Trace	Session Pin	Effort	Ops	Score
A: Config Bundles	4	5	3	3	4	4	3.90
B: Harness Version	4	5	3	3	5	4	4.02
C: Langfuse + AppConfig	5	5	5	5	3	3	4.60
D: Phoenix/AX only	2	1	5	2	4	3	2.67
E: AppConfig only	3	5	3	3	5	4	3.82
F: CDK Hotswap	4	2	3	1	4	4	2.85
G: Braintrust + AppConfig	5	4	4	5	2	2	3.95

Top scorer: Option C (Langfuse + AppConfig) at 4.40 — best if full lifecycle maturity is the goal. Options A and B (native AgentCore) score 3.73 / 3.93 — best if minimal dependencies and fastest time-to-deliver matter. Option D (Phoenix/AX alone) scores lowest (2.68) because it provides zero hot-swap capability.

\$10

Recommended Implementation

Two-phase delivery aligned with the Jira epic's SDK-only scope and minimal delivery requirements.

Phase 1 — MVP (2–3 weeks, SDK primitives only)

Implement Option B (Harness) or Option A (Config Bundles) depending on whether the GENAI Agent Foundry agents use the Harness abstraction or code-based Runtime. Add PromptClient wrapper with OTEL span emission. This satisfies all three minimal delivery requirements with zero external platform dependency.

Week 1: Prompt extraction	Extract all hardcoded system prompts from agent code into prompts/ directory in Git. Create harness.json or Config Bundle definition for each agent. CI schema validation added.	▼
Week 1: SDK wrapper	Write PromptClient class (Section 7) with AppConfig or bundle source. Wire into agent invocation. Test locally with agentcore dev (hot reload available locally).	▼
Week 2: OTEL instrumentation	Add gen_ai.prompt.version to every LLM invocation span. DISABLE_ADOT_OBSERVABILITY=true. Configure OTEL_EXPORTER to Phoenix (dev) and Arize AX (prod). Verify spans visible in Phoenix trace UI with prompt version filter.	▼
Week 2: Session pinning	Implement version pinning at session creation via AgentCore Memory. Write session-scoped prompt fetch helper. Test: deploy V16 mid-conversation, verify running sessions stay on V15.	▼
Week 3: Hot-swap validation	End-to-end test: update prompt in harness/bundle, verify agent picks up within 60s without restart. Test rollback: revert to previous version in < 30s. Test A/B: split traffic via Gateway, observe version distribution in Phoenix traces.	

Phase 2 — Full Lifecycle (Months 2–4, if required by governance)

Add Langfuse for registry, lineage, approval workflow light, and native trace linkage. This upgrades from Option A/B to Option C without replacing Phase 1 infrastructure — AppConfig continues as the delivery layer.

Week 1: Langfuse setup	Deploy Langfuse OSS (self-hosted, Docker Compose or Kubernetes). Import existing prompts. Create production labels. Configure RBAC for team roles.	▼
Week 2: Sync pipeline	Lambda triggered by Langfuse webhook on label change. Syncs production-labeled prompt to AppConfig. Canary deployment strategy configured.	▼
Week 3: CI integration	GitHub Action: on PR to prompts/ → run Phoenix offline eval against golden dataset. Eval score posted as PR comment. Approval gate configured for RAI Officer.	▼
Week 4: Trace linkage	Switch PromptClient from AppConfig source to Langfuse source. Langfuse trace linkage active. Prompt version visible in Langfuse trace UI with native linkage.	▼
Ongoing: Arize AX	Configure Arize AX online evaluation tasks on production traffic. Alert on quality regression by prompt version. Dashboard: compare V15 vs V16 quality over time.	

Files to Create for MVP

File	Description
prompts/customer-support-system/v2.3.1.jinja2	System prompt template with Jinja2 variables

prompts/customer-support-system/metadata.yaml	id, version, owner, model_compatibility, created_at
sdk/prompt_client.py	PromptClient wrapper class (Section 7)
sdk/otel_setup.py	OTel provider config with ADOT disabled, exporter to Phoenix/AX
agentcore.json	Harness definition OR Config Bundle reference
scripts/sync_to_appconfig.py	Lambda/script: publish approved prompt to AppConfig
scripts/validate_prompt_schema.py	CI: validate metadata.yaml schema, detect secrets
.github/workflows/prompt-ci.yml	CI: schema validation + Phoenix offline eval on PR
docs/prompt-update-runbook.md	How to update, rollback, A/B test a prompt. On-call reference.

§11

Acceptance Criteria — Complete

Updated Jira acceptance criteria addressing all requirements, edge cases, and observability.

Minimal Delivery (Phase 1 / In-Scope for This Epic)

AC-1: A new prompt version published to Git and deployed...

A new prompt version published to Git and deployed to AppConfig/Harness/Bundle is picked up by all running agents within ≤ 90 seconds, with no agent container restart.

AC-2: Every LLM invocation span carries gen_ai.prompt.id...

Every LLM invocation span carries gen_ai.prompt.id and gen_ai.prompt.version as OTel span attributes. These attributes are visible and filterable in Phoenix (dev) and Arize AX (prod) trace UIs.

AC-3: Sessions that started on prompt V15 continue using...

Sessions that started on prompt V15 continue using V15 for their entire lifetime after V16 is deployed. Version is stored in AgentCore Memory at session creation.

AC-4: If AppConfig or the bundle endpoint is unreachable...

If AppConfig or the bundle endpoint is unreachable, the agent falls back to the last cached prompt version and continues serving requests. No hard failure on config unavailability.

AC-5: Rollback to any previous prompt version takes effe...

Rollback to any previous prompt version takes effect within ≤ 90 seconds via a single CLI command or API call. No code change or container rebuild required.

AC-6: A/B test between two prompt versions can be config...

A/B test between two prompt versions can be configured via AgentCore Gateway or AppConfig canary strategy. Session assignment is sticky (same session always gets same version).

AC-7: DISABLE_ADOT_OBSERVABILITY=true is set in all Agen...

DISABLE_ADOT_OBSERVABILITY=true is set in all AgentCore agent deployments. OTel traces flow to Phoenix/AX without ADOT conflict.

AC-8: No API keys, credentials, or PII appear in any pro...

No API keys, credentials, or PII appear in any prompt template in Git. CI fails on secrets detection.

Out of Scope for This Epic (Orange Scope / Future)

- TP dashboard UI for prompt editing and review — Orange team scope
- Full prompt registry with semver, lineage graph, dependency tracking (Phase 2)
- Formal approval workflow with RBAC and audit trail (Phase 2)
- RAI validation pipeline and red team testing integration (separate epic)
- EU AI Act AIBOM generation (separate compliance epic)
- Braintrust eval-gated CI/CD quality gates (Phase 2)

Dependencies (Must Be Resolved Before Sprint Planning)

Dependency	Owner	Blocker for
AgentCore Memory provisioned in target environment	Platform team	AC-3 (session version pinning)
AgentCore Config Bundles API access (Preview May 2026)	AWS / Platform team	Option A implementation
Harness GA confirmed available in target AWS region	Platform team	Option B implementation
AppConfig Lambda extension approved for agent containers	Platform / Security	Option E implementation
DISABLE_ADOT_OBSERVABILITY approved by observability team	Observability / SRE	AC-7 (OTel to Phoenix/AX)
Phoenix/AX API keys and OTel endpoint configured in Secrets Manager	Observability team	AC-2 (trace correlation)
Langfuse OSS deployment (if Phase 2 in scope)	Platform team	Phase 2 only

§12

Addressing the Original Ticket —
Line by Line

Mapping research findings back to every item in the GENAIAF-559 Jira ticket.

User Story

'As an Observer, I want to iterate on prompts without redeploying the agent'

Improved version: As a Prompt Engineer, I want to update a system prompt version and have all running agents pick it up within 90 seconds — without container rebuild, without interrupting active sessions, and with the prompt version visible in Phoenix/AX traces for quality analysis.

Minimal Delivery Items — Status

Ticket Item	Research Finding	Recommended Solution
Prompt versioning (git-based, tied to agent config)	Git is correct as authoring source. BUT agent config must reference version label/ID, not inline string. Two native AgentCore mechanisms now exist.	harness.json or bundle definition in Git. Version reference (not content) in agent config. Content served dynamically at runtime.
Hot-swap: runtime prompt switching without redeploy	Three native AgentCore mechanisms: (1) Config Bundle activate, (2) Harness endpoint promotion, (3) AppConfig TTL-based poll. All achieve < 90s without container rebuild.	Option A (bundles) or Option B (harness) for AWS-native MVP. Option C (Langfuse + AppConfig) for full lifecycle.
Prompt version ID emitted in traces	AgentCore's built-in ADOT does NOT emit this automatically. Must be added explicitly as OTEL span attributes in agent SDK code. ADOT must be disabled first.	PromptClient.instrument_span() sets gen_ai.prompt.version on every LLM span. DISABLE_ADOT_OBSERVABILITY=true + OTEL_EXPORTER to Phoenix/AX.

Potential Scope Item — Hot-Swap Without Redeploy?

The ticket listed this as a question under 'Potential Scope'. Research answer: **Yes, fully achievable — and it should be Minimal Delivery, not Potential Scope.**

Config Bundles (Option A), Harness versioning (Option B), and AppConfig (Option E) all deliver true zero-redeploy hot-swap. CDK Hotswap (Option F) is faster deploy but NOT true hot-swap. The distinction matters for the sprint planning estimate.

Challenges and Roadblocks — Complete List

Challenge	Detail	Mitigation
AgentCore Config Bundles in Preview	May change API before GA. Limited documentation. No Terraform/CloudFormation support yet.	Pin to CLI version. Plan for API changes. Use Harness (GA) as fallback.
ADOT conflict with Phoenix/AX	AgentCore injects ADOT at runtime, conflicting with Langfuse/Phoenix SDK HTTP calls. Standard callbacks produce no traces on AgentCore.	DISABLE_ADOT_OBSERVABILITY=true. Configure own OTEL exporter. Verified solution from both AWS blog and Langfuse docs.
AgentCore Memory required for session pinning	Session version pinning requires AgentCore Memory to be provisioned. If Memory is not in the environment, pinning uses DynamoDB fallback (more complex).	Confirm Memory availability early. Budget for Memory provisioning in sprint.

No approval workflow in MVP	Config Bundles and AppConfig have no built-in human approval gate. Anyone with IAM permission can activate a bundle.	CI required fields: peer review PR + eval passing before bundle create. Phase 2 adds formal workflow.
Jinja2 injection risk	Dynamic template rendering with user-supplied variables is a prompt injection vector if variables are not sanitized.	Sanitize all user-supplied variables before Jinja2 render. Restrict which variables come from user input vs internal context.
Bundle/version fan-out at scale	If 50 agent types each have 5 prompt types, you manage 250 bundles/versions. Without a registry, discovery is difficult.	Phase 1: naming convention + DynamoDB catalog table. Phase 2: Langfuse registry for discovery and search.

Appendix: Quick Reference

Option Selection Decision Tree

Q1: Do agents use AgentCore Harness (not code-based Runtime)?	YES → Option B (Harness versioning). UpdateHarness + endpoint promotion. Simplest path. // NO → Q2	▼
Q2: Can the team adopt Config Bundles (Preview May 2026 API)?	YES → Option A (Config Bundles). AWS-native, zero external dependency. // NO → Q3	▼
Q3: Is AppConfig already in use or acceptable to add?	YES → Option E (AppConfig alone) or Option C (Langfuse + AppConfig for full lifecycle). // NO → Q4	▼
Q4: Is full lifecycle management (approval, lineage) required in this sprint?	YES → Option C (Langfuse + AppConfig). // NO → Revisit Config Bundles or AppConfig — avoid CDK Hotswap for hot-swap requirement.	

Hot-Swap Latency Summary

Option	Time to New Prompt Active	Session Safety	Rollback Time
A: Config Bundles	Next invocation after bundle activate (< 1s)	Per-invocation pinning possible	< 5s (activate previous bundle)
B: Harness Endpoint	Next invocation after endpoint update (< 1s)	Session pinning via Memory	< 5s (update endpoint to V-prev)
C: Langfuse + AppConfig	≤ 45s (AppConfig poll cycle)	Session pinning via Memory	< 60s (rollback AppConfig deploy)
E: AppConfig alone	≤ 45s (AppConfig poll cycle)	Session pinning via Memory	< 60s
F: CDK Hotswap	~30s (faster deploy)	No — container restarts	~30s (another hotswap)

OTel Span Attributes Cheat Sheet

```
# Minimum viable attributes for Phoenix/AX correlation
gen_ai.prompt.id      = ''      # Required
gen_ai.prompt.version = ''      # Required

# Enhanced attributes (recommended)
gen_ai.prompt.sha256  = ''      # Tamper detection
gen_ai.bundle.id      = ''      # If using Config Bundles
langfuse.prompt.id    = ''      # If using Langfuse
session.id            = ''      # Required for session analysis
deployment.label      = 'production' # or 'canary', 'staging'
ab_test.variant       = 'A'      # If in A/B test

# OTel environment (set in agentcore.json or CDK)
DISABLE_ADOT_OBSERVABILITY=true
OTEL_EXPORTER_OTLP_ENDPOINT=
OTEL_EXPORTER_OTLP_HEADERS=
```

Resource Links

- **AgentCore Config Bundles docs** — <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/optimization.html>
- **AgentCore Harness versioning** — <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/harness-versioning.html>

- **AgentCore Harness GA blog** — <https://aws.amazon.com/blogs/machine-learning/amazon-bedrock-agentcore-harness-is-now-generally-available-go-from-idea-to-production-grade-agent-in-minutes/>
- **Langfuse + AgentCore integration guide** — <https://langfuse.com/integrations/frameworks/amazon-agentcore>
- **AWS blog: AgentCore + Langfuse observability** — <https://aws.amazon.com/blogs/machine-learning/amazon-bedrock-agentcore-observability-with-langfuse/>
- **Langfuse prompt management get-started** — <https://langfuse.com/docs/prompt-management/get-started>
- **Langfuse prompt → trace link** — <https://langfuse.com/docs/prompt-management/features/link-to-traces>
- **AgentCore Optimization (Config Bundles + A/B)** — <https://aws.amazon.com/blogs/machine-learning/introducing-the-agent-performance-loop-agentcore-optimization-now-in-preview/>
- **CDK Hotswap for AgentCore (Jan 2026)** — <https://dev.to/aws-heroes/aws-cdk-hotswap-deployments-now-support-bedrock-agentcore-runtime-42c7>
- **Caylent: Config Bundles hands-on** — <https://caylent.com/blog/from-prompt-edits-to-performance-loops-hands-on-with-amazon-bedrock-agentcore-optimization>
- **Langfuse existing OTel setup / ADOT conflict resolution** — <https://langfuse.com/faq/all/existing-otel-setup>
- **Arize Phoenix LiteLLM prompt integration** — https://docs.litellm.ai/docs/proxy/arize_phoenix_prompts